Arrays

What they are, why they matter, and where they are used

1. What is an Array?

An **array** is a collection of elements of the **same data type** stored in **contiguous (side-by-side) memory locations** and accessed through a numeric position called an **index**. Instead of creating ten separate variables to hold ten student scores, we create one array that holds all ten.

In most languages, including C++, indexing starts at **0**. An array of 5 integers looks like this in memory:

Index	0	1	2	3	4
Value	72	85	90	64	78

Declaring and using an array in C++

```
int scores[5] = {72, 85, 90, 64, 78};   // declare + initialise

cout << scores[2];        // prints 90 (third element)
scores[3] = 70;           // update the fourth element

int sum = 0;
for (int i = 0; i < 5; i++)   // visit every element
    sum += scores[i];
```

Common types of arrays

- **One-dimensional (1D):** a simple list, e.g. the scores of one student.
- **Two-dimensional (2D):** a table of rows and columns, e.g. scores[student][subject]. Used for grids, matrices and spreadsheets.
- **Multi-dimensional:** three or more dimensions, e.g. colour images (height x width x RGB channel).
- **Dynamic arrays:** arrays that can grow or shrink at run time (std::vector in C++, ArrayList in Java, list in Python).

2. Why Are Arrays Necessary?

Without arrays, programmers would need a separate variable for every single value, which is impractical, unreadable and impossible to scale. Arrays solve this and provide several key benefits:

- **Manage many values with one name.** Storing 1,000 student marks needs one array, not 1,000 variables.
- **Fast random access.** Because elements sit next to each other, the computer finds element *i* with simple arithmetic (start address + i x element size). Reading `scores[500]` takes the same time as `scores[0]`. This is called O(1) access.
- **Easy processing with loops.** A single loop can sum, search, sort or update every element, so code stays short and clean.

- **Memory efficiency and cache friendliness.** Contiguous storage has no extra overhead per element and works well with the CPU cache, which makes array traversal very fast.
- **Foundation for other data structures.** Stacks, queues, heaps, hash tables and matrices are all commonly built on top of arrays. Many algorithms (binary search, sorting, dynamic programming) rely on them.

Limitations to keep in mind

- A standard array has a **fixed size** that must be decided up front.
- Inserting or deleting in the middle requires **shifting** elements, which is slow for large arrays.
- All elements must be of the **same type**.
- C++ does not check bounds automatically, so reading `scores[10]` in a 5-element array is undefined behaviour.

3. Real-World Applications of Arrays

Application	How arrays are used
Gradebooks and student records	Scores are stored in a 2D array (rows = students, columns = subjects). Loops compute totals, averages, highest and lowest marks, and rank students.
Spreadsheets (Excel, Google Sheets)	A sheet is essentially a 2D array of cells. Formulas such as SUM and AVERAGE loop over ranges of that array.
Images and video	An image is a 2D (or 3D) array of pixels. Filters like blur or brightness change the value of each array element; video is a sequence of such arrays.
Music and audio players	Audio is stored as an array of samples taken thousands of times per second. Playlists are also arrays of songs.
Games	Game boards (chess, tic-tac-toe, Sudoku, Minesweeper) are 2D arrays; high-score tables and inventories are 1D arrays.
Databases and inventory systems	Records are held in arrays in memory while being searched, sorted or filtered before display. Stock lists and price lists rely on this.
Maps, GPS and navigation	Road networks and city grids are represented with arrays (adjacency matrices or lists) so shortest-path algorithms can run on them.
Machine learning and scientific computing	Data sets, neural-network weights and matrices are all large numeric arrays (e.g. NumPy arrays).
Operating systems and networking	Process tables, memory pages, and packet buffers are managed as arrays for quick indexed access.
Social media and e-commerce feeds	Posts, comments, products and search results are stored in ordered arrays that are sorted and displayed to you.

4. Case Study: A Gradebook Application

To show arrays in action, we will build a **Student Gradebook** in C++. It uses a 1D array of names, a 2D array of scores (students x subjects), and loops to calculate totals, averages, grades, class statistics and rankings, then saves a report to a file. The full source code and instructions are in the

accompanying GitHub repository.

Conclusion

Arrays are one of the most fundamental data structures in programming. They let us store and process large amounts of related data efficiently, power everyday software from spreadsheets to games, and form the building blocks for more advanced structures and algorithms. Understanding arrays well is the first step to writing efficient programs.